

Solutions to Assignment 1

Network Science

PHYS 7332, Fall 2025

Peiwen Liu*

1. *Question 1.*

(a) Advantages:

- CSV:

1. Human readable and simple. CSV files can store networks data in format of edge list and adjacency list.
2. CSV files can be easily parsed by pandas in python, SQL.

- Pickle:

1. Pickle preserves the entire NetworkX object structure and python types.
2. Very convenient if the workflow is entirely python

- GML:

1. Text-based, standardized graph format supported by networkx, gephi, igraph etc. Good for graph analysis and visualization tools.
2. Can encode both graph topology and metadata (node/edge attributes).

- Elasticsearch:

1. a full-fledged search and indexing engine that can handle very large datasets efficiently.
2. support graph queries, aggregation, and filtering
3. enables distributed storage and retrieval (horizontal scaling)
4. can integrate with visualization dashboards (Kibana)

(b) Disadvantages

- CSV:

1. No native enforcement of schema → risk of inconsistent formatting.
2. Not efficient for very large networks (files become huge, parsing is slow).
3. Limited ways to store complex attributes, hierarchies...

- Pickle:

1. Python-only. Not portable to other languages without conversion.
2. Not human-readable

- GML:

1. larger file sizes
2. Limited support for certain data types that everything coerced into strings, ints, or floats.

- Elasticsearch:

1. Complex setup and maintenance compared to flat files.
2. Requires a running server cluster (resource-intensive).

* **ADD CLASS REPO FORK** https://github.com/AerosolKiller/phys7332_fa25.git

(c) Security risks:

Pickle is the main security concern. Pickle files can execute arbitrary Python code during deserialization. If you load a malicious pickle from an untrusted source, it could run harmful code on the machine.

(d) Optimal Scenarios: - CSV:

Best when you want portability and lightweight storage.

Example: Exchanging an edge list between collaborators who use different tools (Python, R, Gephi).

- Pickle:

Best for fast internal use in Python-only workflows.

Example: Saving intermediate results in a long computation pipeline where speed matters more than portability.

- GML:

Best for graph analysis and visualization workflows across different platforms.

Example: Exporting a networkx graph to load into Gephi for visualization, while keeping node attributes intact.

- Elasticsearch:

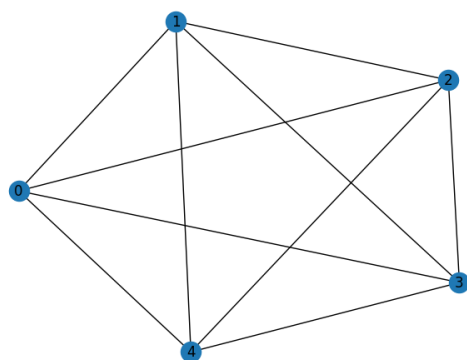
Best for large, dynamic, or production-level applications with complex querying needs.

Example: Storing clickstream or social media interaction data where analysts want to query subgraphs in near real-time and connect it with dashboards.

2. Question 2.

- (a) We cannot perform edge exchange if we have a complete network. The most simple example is a triangle. Edges AB, BC, AC are not independent. So we will never be able to find a valid pair.

Also, even we find two independent edges in a complete graph (simple graph), we still cannot perform edge exchange because the new formed edges are colliding with existing edges.



- (b) see problem2.ipynb

- (c) **Preserving connectivity.** If we require that the randomized graph remain connected, then many degree-preserving swaps must be rejected. In particular:
- Fewer valid swaps: Many two-edge swaps would break a bridge or disconnect an articulation point. These must be disallowed.
 - Costly checks: Each tentative swap must be followed by a connectivity check (e.g., BFS/DFS or strongly connected component test), which is $O(n + m)$.
 - Higher failure streaks: Because far more attempts are rejected, the algorithm will reach its failure limit quickly unless we allow a very large number of retries or relax the connectivity requirement.

This makes edge exchange substantially harder: the state space of valid graphs shrinks, and the randomization mixes more slowly.

- (d) **Preserving attribute correlation.** Suppose every node i has a binary attribute $x_i \in \{-1, +1\}$. Then every edge corresponds to an ordered (directed) or unordered (undirected) attribute pair. To preserve correlation, any edge exchange must yield new edges with the same attribute pairs as the originals. For example, an undirected edge between two $+1$ nodes must always be replaced by another edge between two $+1$ nodes.

This makes edge exchange more difficult because:

- The pool of compatible partner edges shrinks, especially in networks with strong homophily or imbalanced attribute distributions.
- Many candidate swaps that preserve degree and connectivity must still be rejected because they violate the attribute-pair requirement.
- In directed graphs, preserving *ordered* attribute pairs is even more restrictive, further increasing rejection rates.

Thus, enforcing attribute correlation greatly reduces the number of allowable moves and slows mixing.

3. Question 3.

- (a) see problem3.ipynb section p3(a)
 (b) see problem3.ipynb section p3(b)
 (c) I compared my implementation of the unscaled betweenness centrality with the reference across a collection of small graphs designed to expose edge cases. For networkx I used the options `normalized=False` and `endpoints=False` to match our convention.

Results.

- i. *Path graph P_5 .* The function produced larger values than networkx at interior nodes (e.g. differences of $+3.0$ at node 1, $+4.0$ at node 2, $+3.0$ at node 3). This indicates a discrepancy in how contributions are counted on unique shortest paths in trees.

- ii. *Directed acyclic graph (DAG)*. With edges $(0, 1), (1, 2), (2, 3), (1, 3)$, both implementations matched exactly. Here, the uniqueness of shortest paths avoided any ambiguity.
- iii. *Cycle C_4* . In the 4-cycle, opposite corners are connected by two distinct shortest paths. Our method assigned each node an extra +0.5 compared to `networkx`. This shows that in the presence of multiple geodesics, our counting rule differs from the fractional treatment in `networkx`.
- iv. *Diamond graph*. For the undirected diamond $s-x-v-t$ and $s-y-v-t$, the node v lies on all shortest $s-t$ paths. Our code gave higher centrality to v and also shifted weight to x and y compared to `networkx` (differences: 0.5, 1.0, 1.0, 2.5 across nodes). This reflects a systematic divergence: when multiple shortest paths overlap at a node, our indicator rule double-counts contributions rather than distributing them fractionally.
- v. *Disconnected graph $K_2 \cup K_2$* . Both methods produced mostly zero values, effectively ignoring unreachable pairs. Our implementation additionally returned informative messages, warning that ignoring unreachable pairs biases centrality toward larger or denser components, and reporting the number of unreachable ordered pairs (eight in this case).

Discussion. The two implementations coincide when:

- shortest paths are unique (trees, DAGs),
- or when unreachable pairs are consistently ignored (disconnected graphs).

Differences arise when multiple equally short paths exist. `networkx` assigns each path a fractional weight so that their contributions sum to one, whereas our indicator-based method increments each path in full. Thus, our centrality values are systematically larger at nodes contained in multiple geodesics. In practice, this shows that subtle choices in how to apportion contributions can significantly change centrality values, even though the definitions are superficially similar.

(d) Robustness and correctness tests

Goal. I built a test harness `test_centrality` to exercise a centrality routine under edge cases (robustness) and small graphs with known answers (correctness). The harness accepts a callable `cfunc(G, **kwargs)` (here, the `indicator_betweenness_unscaled`) and forwards options such as `endpoints` and `require_connected` when supported.

Method. To make the tester generic, I use a guarded caller that attempts to pass optional keywords and silently drops unsupported ones (to avoid false negatives from `TypeError`). Then run a fixed battery of graphs and record, per test, whether (i) exceptions are raised as expected, and (ii) numeric results match the expected values up to a tolerance `tol`.

Correctness cases.

- i. *Triangle K_3* . No interior nodes $\Rightarrow$ all zeros.

- ii. *Path* P_5 (0–1–2–3–4). Unique geodesics; unscaled, unweighted, `endpoints=False` counts are

$$\{0:0, 1:6, 2:8, 3:6, 4:0\}.$$

- iii. *Star* S_5 (center 0). All ordered leaf–leaf pairs go through the center: center = 12, leaves = 0.

- iv. *Cycle* C_4 . By symmetry, all nodes equal.

- v. *Directed chain* $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$. With `endpoints=False`:

$$\{0:0, 1:2, 2:2, 3:0\}.$$

Results (exact run).

- *Non-graph input*: OK — raised `TypeError` with a clear message.
- *Empty graph*: OK — raised `CentralityError` (“Empty graph: no nodes; ...”).
- *Single node*: OK — value 0.0.
- *Edgeless graph*: OK — all zeros.
- *Disconnected (two comps)*: OK — finite values, e.g., $\{0:0.0, 1:2.0, 2:0.0, 10:0.0, 11:0.0\}$ plus informative warnings.
- *Directed not weakly connected*: OK — finite values, e.g., $\{0:0.0, 1:1.0, 2:0.0, 10:0.0, 11:0.0\}$.
- K_3 : OK — all zeros.
- P_5 : OK — exact match to expected counts.
- S_5 : OK — center 12, leaves 0.
- C_4 : OK — uniform values.
- *Directed chain*: OK — exact match to expected counts.

A compact status table:

Test	Status
<code>non_graph_input</code>	OK (raised <code>TypeError</code>)
<code>empty_graph</code>	OK (raised <code>CentralityError</code>)
<code>single_node</code>	OK (0.0)
<code>edgeless_graph</code>	OK (all 0)
<code>disconnected_graph</code>	OK (finite; warns; e.g. $\{0:0, 1:2, 2:0, 10:0, 11:0\}$)
<code>directed_not_weakly_connected</code>	OK (finite; e.g. $\{0:0, 1:1, 2:0, 10:0, 11:0\}$)
<code>K3_triangle_all_zero</code>	OK
<code>path_P5_exact_counts</code>	OK (match)
<code>star_S5_center_12_leaves_0</code>	OK (match)
<code>cycle_C4_uniform</code>	OK (uniform)
<code>directed_chain_exact</code>	OK (match)

Conclusion. All robustness and correctness checks pass (OVERALL: PASS). The harness verifies clear error messages on invalid inputs and exact agreement with hand-computable cases. Disconnected cases are handled gracefully (unreachable

pairs contribute 0), and the tester reports informative diagnostics (e.g., warnings and example value maps) to aid debugging.

4. Question 4.

- (a) See code in problem4.ipynb section P4(a)
- (b) See code in problem4.ipynb section P4(b)

	Department	n_{courses}	Avg. prerequisites
	English	114	3.474
(c)	Creative Writing	18	3.333
	Anthropology	36	3.222
	English Writing	19	3.158
	Psychology	97	2.495

The English department has the highest average number of prerequisites per course (≈ 3.47). This is somewhat surprising, since one might expect more technical departments such as STEM fields to have longer prerequisite chains. However, English curricula often have tiered sequences of writing and literature courses, leading to this high average.

- (d)
 - i. BIOL 1107 — Foundations of Biology
 - ii. BIOL 1113 — General Biology 2
 - iii. PHSC 2301 — Human Physiology 1
 - iv. PHSC 2303 — Human Physiology 2
 - v. HSCI 1105 — Human Nutrition
 - vi. NRSB 2220 — Health Assessment and Fundamental Nursing Skills
 - vii. NRSB 3320 — Nursing Care of Adults 1
 - viii. NRSB 3420 — Nursing Care of Adults 2
 - ix. NRSB 4502 — Nursing Care of the Child
 - x. NRSB 4995 — Comprehensive Nursing Practicum

This indicates that students in Nursing must progress through a long, structured sequence of courses, reflecting the cumulative skill requirements of the discipline. The chain length is 9.

Note: I did find self loops in Spanish but that is because they are "Placement" in the prerequisites.

- (e) Both networks are derived from the course catalog but capture different structural properties:
 - **Course prerequisite network:** This network is directed and acyclic (ideally). It encodes hierarchical dependencies among courses. Research questions here might include:
 - What is the longest path (chain) of required courses in a curriculum?

- Which courses are central “gatekeepers” that unlock many advanced courses?
- How modular is the curriculum (e.g., are there clusters of courses that mostly depend on each other)?
- **Word co-occurrence network:** This network is undirected and typically much denser. It reflects semantic relationships between words in course descriptions. Research questions here might include:
 - Which terms or topics are most central in describing a department’s curriculum?
 - How do clusters of words reveal thematic groupings (e.g., data science, global health, creative writing)?
 - Are there cross-departmental similarities in language use?

In summary, the prerequisite network is best suited for studying curricular structure and progression, while the co-occurrence network is better for exploring thematic content and the semantic landscape of the catalog.

5. *Question 5.*

- (a) Nodes are the users on tweeter. Links are the retweet actions. The data set is from <https://networkrepository.com/rt-bahrain.php>

This is a retweet network which is collected from “Bahrain” hashtag on tweeter.

- (b) The degree distributions of both the empirical and randomized retweet networks are nearly identical, as expected from a degree-preserving rewiring null model. Both show a heavy-tailed form, where most nodes have small in- or out-degree, and a few nodes act as hubs with very high connectivity. This overlap confirms that the randomization successfully preserved degree heterogeneity, meaning the presence of hubs and the skewed distribution of activity levels are not artifacts of the randomization process. However, because the degree sequence is fixed by construction, this measure alone cannot reveal deeper structural differences between the empirical and null networks.

The clustering coefficient distributions appear broadly similar between the empirical and randomized networks. Both follow a decaying trend, where higher clustering values become progressively less likely. This suggests that, once degree sequence is fixed, much of the observed clustering can be explained without invoking additional structure.

Betweenness centrality distributions also highlight key differences. In the empirical network, the distribution is more skewed, with a heavier tail—indicating the presence of broker nodes that occupy critical positions in information flow. These are likely to be influential accounts, such as news outlets or prominent individuals, that bridge different parts of the network and channel a large fraction of retweet

traffic. In contrast, the randomized null model disperses paths more evenly across nodes, reducing the concentration of betweenness.

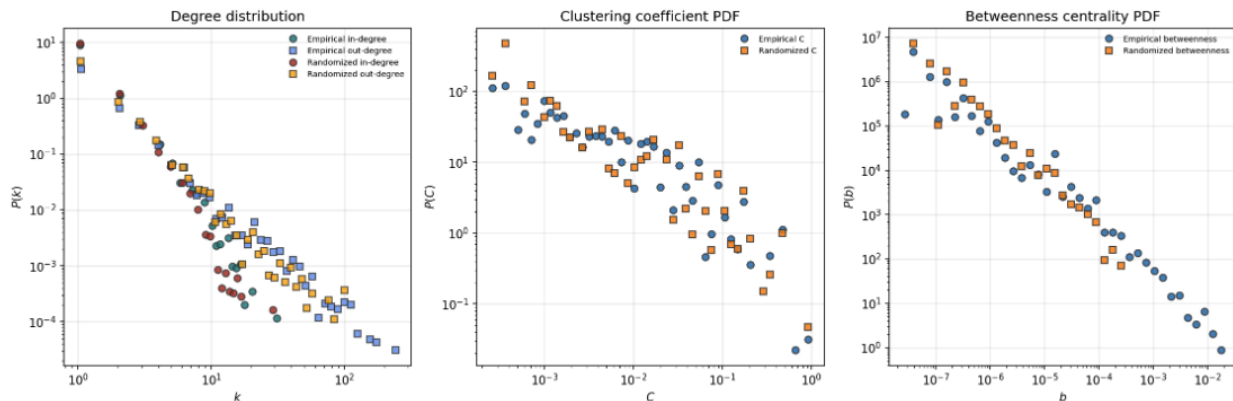


FIG. 1. Degree distribution, Clustering coefficient, and betweenness centrality measurements

- (c) When comparing the empirical retweet network to its randomized counterparts, some measures behave similarly while others reveal important differences. The degree distributions are identical across both cases, as expected, because the null model preserves in- and out-degrees by construction. The clustering coefficient distributions also appear broadly similar, suggesting that once degree heterogeneity is fixed, the level of local clustering can largely be explained by degree sequence alone. However, the betweenness centrality distributions differ: in the empirical network, a small set of nodes occupy disproportionately high-betweenness positions, indicating the presence of brokers or influential accounts that connect different user groups. In contrast, the randomized networks distribute paths more evenly across nodes, diluting the dominance of any particular intermediaries.

6. (Optional) Question 6 (5 bonus points).

For every problem set this semester, there will be an opportunity to earn extra credit. Since this is a new course, we are eager to receive genuine feedback on the material, pace, instruction style, etc. of the course so far.

I've found this course to be very engaging and valuable as a beginner in network science. The balance between theory and coding implementation has been helpful, and I feel that the hands-on exercises are training me to become more comfortable with the technical side of the field. That said, one area I think could be improved is providing more context before diving into implementation. For example, short pre-class readings or overviews explaining why we are focusing on particular concepts or algorithms would help me better understand the motivation behind the tasks. This additional context would make the coding exercises feel more grounded in the bigger picture of network science, rather than just technical practice.

If you reference anything, add it `main.bib` and cite using `\cite{Hartle2020wegd}`.
